

P2PFL SVM Results

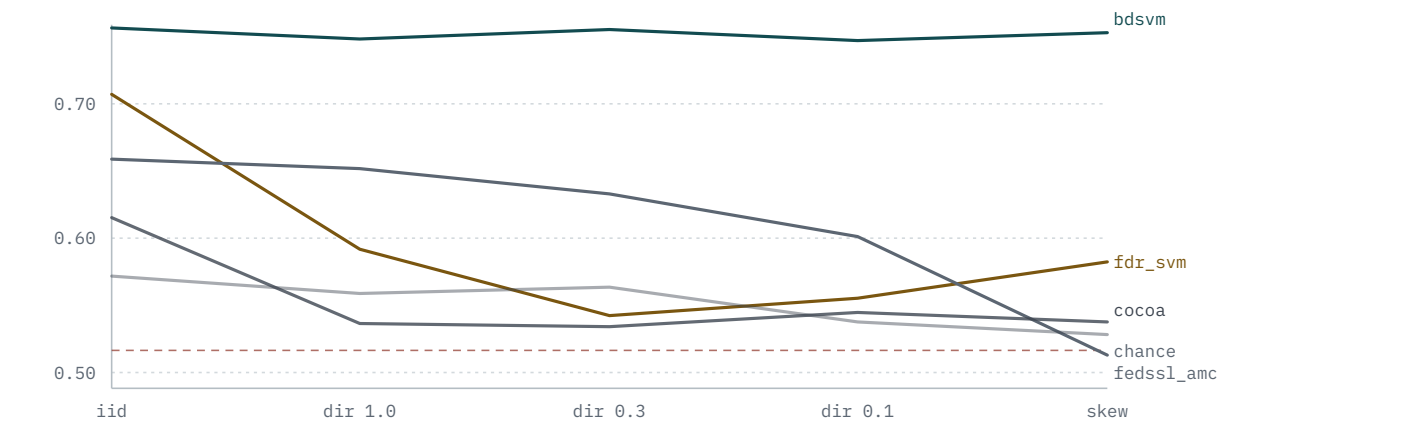
Six FL algorithms as PeerSim gossip protocols, measured against their own server-based baselines. One method keeps its accuracy intact across every partition; the rest collapse toward chance. The reason is structural, and it survives decentralisation.

216 CONFIGURATIONS	0 FAILURES	0.0045 BDSVM NON-IID GAP	0.1457 WORST NON-IID GAP
--	----------------------------------	--	--

The headline: BDSVM is flat, everything else is not

covtype, 10 nodes, 100 cycles, connected overlay (k=2), averaged over ring / random_kout / full. The last column is the drop from IID to label_skew — the property the whole non-IID literature is about.

METHOD	IID	DIRICH 1.0	DIRICH 0.3	DIRICH 0.1	LABEL_SKEW	DROP	SERVER
bdsvm	0.7276	0.7190	0.7262	0.7176	0.7231	0.0045	0.7374
fedavg_svm	0.5432	0.5295	0.5348	0.5097	0.5000	0.0432	0.6772
cocoa_plus	0.5684	0.5166	0.5079	0.5096	0.5108	0.0576	—
cocoa	0.5862	0.5071	0.5054	0.5153	0.5078	0.0785	—
fdr_svm	0.6784	0.5635	0.5139	0.5268	0.5536	0.1248	0.6839
fedssl_amc	0.6298	0.6225	0.6036	0.5724	0.4841	0.1457	0.5582
centralized	single-machine upper bound						0.7577
majority class	chance level						0.5159



covtype, k=2, three topologies averaged. BDSVM tracks flat; the rest converge on the chance line.

Why BDSVM holds and the others don't

Not a tuning artefact. BDSVM's aggregation is an identity, and identities do not care how the data was split.

The aggregation is exact, so the partition cannot matter

Every worker sends $C_m = K'^T_m D_a^m K'_m$ and $d_m = K'^T_m D_a^m y^m$, and the solve is over $(\sum_m C_m + K'^T_p)$. That sum equals the centralized matrix exactly — measured at $4.6e-16$ relative error for 10-node IID, 10-node dirichlet 0.1 and 50-node label_skew alike. There is no drift term to correct because there is no drift.

The other five aggregate *models*. A model trained on one class is not a partial answer to the global problem, so averaging it in is destructive — which is exactly the shape of the table above.

The catch: exactness needs every node to be reachable

WireKOut with $k=1$ gives each node one out-edge, and an undirected random 1-out graph is usually disconnected — at $n=10$, seed 42, it splits 7+3. Contributions cannot cross a component boundary, so BDSVM's contribution table topped out at 3–7 of 10 origins and its aggregate was 41% off. No number of cycles or bytes fixes that: we tried 30→100 cycles and 2→10 table entries per message (10× the bandwidth) and the gap moved by 0.02.

Connectivity is now recorded in every CSV row, so this is auditable rather than inferred.

WHAT CONNECTIVITY BUYS, BY METHOD — RANDOM_KOUT, K=1 → K=2

METHOD	IID	DIRICHLET 0.1	LABEL_SKEW
bdsvm	0.7150 → 0.7203 (+0.005)	0.6476 → 0.7192 (+0.072)	0.6494 → 0.7092 (+0.060)
fedssl_amc	0.5601 → 0.6464 (+0.086)	0.5468 → 0.6241 (+0.077)	0.4841 → 0.4841 (0.000)
cocoa	0.6169 → 0.6208 (+0.004)	0.5072 → 0.5262 (+0.019)	0.5000 → 0.5233 (+0.023)
fdr_svm	0.6803 → 0.6732 (−0.007)	0.5188 → 0.5331 (+0.014)	0.5488 → 0.5666 (+0.018)
fedavg_svm	0.5655 → 0.5314 (−0.034)	0.5100 → 0.5129 (+0.003)	0.5000 → 0.5000 (0.000)
cocoa_plus	0.6354 → 0.5375 (−0.098)	0.5000 → 0.5064 (+0.006)	0.5000 → 0.5324 (+0.032)

Read the top row: connectivity is nearly free under IID (+0.005) and worth seven accuracy points under skew (+0.072). Under IID any subset of shards is a representative sample, so a partial sum is a scaled version of the right answer; under label skew a missing origin is a missing class. Only the method that needs exact aggregation is sensitive to this, which is the same mechanism seen from the other side.

CoCoA+ moving -0.098 the wrong way is consistent with its known instability: more aggregation is worse for the additive variant, because gossip re-counts progress that neighbours already share.

rcv1 reorders everything

$d = 47,236$, 20k samples, 30 cycles, random_kout $k=2$. The ranking from covtype does not carry over, and the communication column is where BDSVM's story turns.

METHOD	IID	DIRICHLET 0.3	LABEL_SKEW	MB / NODE
fdr_svm	0.9174	0.8717	0.8355	11.3
cocoa	0.9392	0.8583	0.7407	11.3
cocoa_plus	0.9292	0.8536	0.7556	11.3
bdsvm	0.8561	0.6607	0.7909	956.5
fedssl_amc	0.8345	0.8983	0.5202	217.7
fedavg_svm	0.8038	0.6356	0.5020	11.3

BDSVM's budget has to scale with the dimension, and messages are quadratic in it

At $P=100$ — the value that works for covtype — rcv1 sits at chance. The budget fixes the model's capacity, and 100 random basis vectors cannot span a 47k-dimensional space:

$P = 100 \rightarrow 0.5715$ message 0.1 MB

$P = 400 \rightarrow 0.7738$ message 1.3 MB

$P = 1000 \rightarrow 0.8572$ message 8.0 MB

$P = 2000 \rightarrow 0.8998$ message 32.0 MB

A message is $(P+1)^2$ floats. So on high-dimensional sparse data BDSVM's communication advantage disappears exactly where its accuracy starts needing P to grow — 956 MB per node against 11.3 MB for the primal methods, for less accuracy than fdr_svm. This is a real limit of the method, worth stating.

Consensus, and what it costs

covtype, IID, $k=2$ — accuracy / consensus error $\|w_i - \text{mean}(w)\|$.

METHOD	RING	RANDOM_KOUT	FULL	MB / NODE
bdsvm	0.7205 / 11.38	0.7203 / 6.85	0.7421 / 2.52	32.90
fdr_svm	0.6830 / 0.20	0.6732 / 0.21	0.6788 / 0.20	0.04
fedssl_amc	0.6201 / 0.00	0.6464 / 0.00	0.6230 / 0.00	0.76
cocoa	0.5275 / 0.95	0.6208 / 0.40	0.6104 / 0.10	0.04
cocoa_plus	0.5000 / 7.31	0.5375 / 2.79	0.6676 / 0.25	0.04
fedavg_svm	0.5540 / 6.31	0.5314 / 5.88	0.5442 / 6.11	0.04

fdr_svm reaches the tightest consensus of any method at the lowest cost, and holds it across every topology — the ADMM dual variable is doing real work. fedavg_svm's ~6.0 barely moves with topology, because it is dominated by the size of the post-merge local step rather than by gossip: consensus error tracks $\text{eta}_1 = 1/\lambda$ almost exactly ($1e-4 \rightarrow 530$, $1e-2 \rightarrow 6.1$, $1.0 \rightarrow 0.18$). fedssl_amc's exact zero is the encoder having frozen and all nodes agreeing on the classifier.

Open, and honest about it

FedSSL-AMC cannot learn under label_skew

0.4841 on covtype, 0.5202 on rcv1 — chance. Every node holds one class, so no node can fit a local SVM in phase 2. The server version fails the same way (0.5012), so this is the algorithm, not the port. Decide whether to report the limitation or drop the cell before the numbers go in a table.

CoCoA+ peaks then degrades on long runs

Lowering gamma delays the collapse rather than preventing it (gamma=0.5 peaks at 30 cycles, 0.25 at 60, 0.1 at 120). Gossip breaks the primal-dual correspondence: a node adds neighbours' increments at full strength but its own alpha only covers its own samples. Reportable as a finding about the additive variant; fixing it is research, not a bug fix.

Three defaults that were wrong, now measured

gamma = 1/n_features made the rcv1 kernel constant to within 2e-07. Now the median heuristic. **WireKOut k=1** left the overlay disconnected; k is now a grid dimension and connectivity is recorded. **Dense pre-images** are near-orthogonal to every document in 47k dimensions, so distances concentrate into [1.957, 2.039]; a sparse option exists but the concentration is dimensional, not fixable by sampling.

216 configurations, 0 failures. covtype: 10 nodes, 100 cycles, lambda 1e-2. rcv1: 10 nodes, 30 cycles, lambda 1e-4, P=1000 sparse pre-images for bdsvm.

Every local solver verified numerically identical to its server counterpart (7.2e-08 or better), so aggregation is the only difference between the two settings.

Raw data: results/grid_covtype.csv, results/grid_rcv1.csv on branch Lincoln.